

SysMLv2 Systems Engineering Methodology Skill

This skill file equips AI Agents (like Antigravity, Claude, ChatGPT) with the knowledge of how to guide a user through standard Systems Engineering methodologies (INCOSE, OOSEM, MagicGrid) and subsequently translate that gathered knowledge into a robust SysMLv2 model.

Agent Instructions: How to use this skill

1. **Identify the User's Methodology:** When a user asks to build a SysMLv2 model, ask them if they want to follow a specific methodology:
 - **INCOSE Process** (Standard SE Lifecycle)
 - **OOSEM** (Object-Oriented Systems Engineering Method)
 - **MagicGrid** (Dassault/Cameo 3x4 matrix approach)
2. **Provide the Template:** Once the user selects a methodology, output the corresponding Markdown template (found in `LLM_skills/`) to the user. Ask them to fill it out as completely as possible.
 - Use `incose_template.md` for INCOSE.
 - Use `oosem_template.md` for OOSEM.
 - Use `magicgrid_template.md` for MagicGrid.
3. **Gather Information:** The user will provide the filled-out template. Review it to ensure the problem space, logical architecture, and physical implementation (if applicable) are clear. Ask clarifying questions if sections are contradictory.
4. **Translate to SysMLv2:** Use your primary SysMLv2 language skill (from `LLM_skills/skill.md`) to translate the gathered template information into valid SysMLv2 code.
 - Pay special attention to the *SysMLv2 Mapping* hints provided in the templates.
 - Ensure strict separation of concerns (e.g., Logical vs Physical `part defs`) as dictated by the chosen methodology.
 - Implement robust traceability using `satisfy` (requirements) and `allocate` (architecture) relationships.

Key Methodology Principles for SysMLv2

- **Requirements Traceability:** Every methodology requires that system elements trace back to stakeholder needs. In SysMLv2, ensure you use `satisfy` from the structural/behavioral elements to the `requirement usages`.
 - **CRITICAL RULE:** All requirements intended as requirements of the system MUST be usages (e.g., `requirement <'ID'> 'Short Name' { ... }`), NOT definitions (`requirement def`).
 - Only use `requirement def` when defining a specific *kind* of requirement (e.g., `requirement def PerformanceRequirement`).
- **Logical vs. Physical:** OOSEM and MagicGrid heavily rely on separating Logical architectures (technology agnostic) from Physical/Implementation architectures (technology specific). Model this in SysMLv2 by creating distinct `part def` hierarchies and using `allocation` to map Physical parts to Logical parts.
 - **CRITICAL RULE:** `allocate` only works between **usages**, not definitions. You must instantiate your physical and logical `part defs` as `part usages` before allocating them.

- **Behavior Allocation:** Ensure that `action defs` (behaviors) are explicitly performed by structural elements (using `perform` or by nesting them within the `part def`).
- **Scalar Value Imports:** Any use of scalar values like `Real`, `Integer`, `String`, or `Boolean` must include a private import, e.g., `private import ScalarValues::Real;` or `private import ScalarValues::*;`
- **Views and Layouts:** Always generate a `Views` package including tables and various view types. Ensure you use the `expose` parameter to auto-display the context. Furthermore, always apply `EssentialElementsFilter` and `NonStandardLibraryElementFilter` to custom views to ensure proper layout generation.